Monte Carlo Simulation of a Simple Polymer Chain

Implementation Report

Project_2026_II

12 April 2026

Contents

0.1 Project Overview

In this assignment, our goal was to develop a Monte Carlo simulation program to explore the conformational landscape of a 500-Carbon linear polymer chain (polyethylene) with varying torsional angles. The simulation generates the initial conformation (with & without Hydrogens), measures torsional energy rotations, end-to-end distance, and intramolecular Lennard-Jones interactions, and finally analyzes & visualizes the results.

After creating a serial processing version of the program, parallelization techniques involving both MPI and OpenMP were incorporated. These were compared to the serial version to demonstrate both their individual and combined High Performance Computing capabilities & benefits.

Many techniques taught in class were crucial in the development of this program, including: - **Git/Github** as a code respository & teamwork coordination platform - **Makefile** to automate the compilation and execution of the code - **Shell scripting** for facilitating server deployment - **MPI & openMP** to parallelize the code - **Python** for post-processing and plotting

0.2 Repository Structure

Github was used as our team's code repository management system, and from the root of the project, the code is divided among 3 main folders: `src`, `results`, and `docs`.

- `src` contains:
  - Makefile & shell scripts
  - Various versions of the main program (serial & parallel)
  - A `confs` folder where initial configurations are stored and compile-time options can be controlled via a file called `input.dat`
  - A `lib` folder where module source files, python analysis files, and plotting style dependencies reside (*further detail below*)
- `results` contains the output data from the simulations
- `docs` contains documentation and reports, including `img` folder for embedded images

In addition to these folders, a customary `README` file is included in the root directory detailing run instructions and code dependencies/requirements. The bulk of the code is written in Fortran, while the post-processing is done in Python.

Module Library Introduction The following modules support the *`main...f90`* files: - `energy.f90` & `energy_all_atoms.f90` - Permit **dual energy modes**, computing energetic contributions using either TraPPE-UA or OPLS-AA (with effective backbone potentials) depending on the `explicit_h` toggle dynamically read from `input.dat`. - `initial-conf.f90` - Generates initial configuration of the polymer chain. - `io.f90` - File input/ouput module. - `monte_carlo.f90` - Provides the rotation algorithm and subroutine MCS step used to update spatial configurations. - `observables.f90` - Computes the structural observables: End-to-end distance (R_{ee}), Radius of Gyration (R_g), and torsion angles. - `parameters.f90` - Stores global parameters abstracted from *`main...f90`* code. Works in tandem with `input.dat` settings. - *`plot_...py`* files - Various scripts used for plotting observables and serial vs parallel code comparisons. - `requirements.txt` - List of python module dependencies. - `science.mplstyle` - stylized document used in plotting to control visualization theme.

0.3 Code Management via Github

The project leader was responsible for reviewing all pull requests, ensuring that the code followed the project guidelines, and merging the code into the main branch. The team coordinated through a WhatsApp group chat and would regularly check in on each other's progress. Everyone was informed when code was merged so we could all pull the latest changes.

Most times, approving and merging was straightforward with no issues, but there was an interesting challenge when multiple uploads from different team members overlapped, causing conflicts. This forced us to learn about stashing and rebasing to the newly updated master branch before recommitting. The following process was ironed out to ensure a smooth integration:

1. Stash local (uncommitted) changes

   [] git stash

2. Pull latest changes from master branch

   [] git checkout master git pull main master

3. Rebase local changes onto master branch so it starts after the new merged code

   [] git checkout <fork-feature>/<fork-branch> git rebase master

4. Resolve any conflicts & re-apply stashed changes

   [] git stash pop

5. Commit and push (using `--force` parameter due to rebase)

   [] git push <fork-remote> <fork-branch> --force

0.4 Makefile

As the project evolved, so did the Makefile. Here are some of the features we implemented:

Starting off, there are few main variables that control the compilation process.... [] FC = gfortran FFLAGS = -O2 -Wall -Wextra OBJ_DIR = ../bin EXE = $(OBJ_DIR)/main_serial.x LIB_OBJ=(OBJ_DI \ $(OBJ_DIR)/io.o\ ...MAIN_OBJ=(OBJ_DIR)/main_serial.o

NP ?= 4 OMP_THREADS ?= 2

- `FC`: Stands for "Fortran Compiler". We tell it to use gfortran.
- `FFLAGS`: These are the compiler flags. `-O2` is the optimization schema, and `-Wall -Wextra` enables warnings.
- `OBJ_DIR` & `EXE`: Defines the compiled binaries directory and the name of the final executable.
- `LIB_OBJ` & `MAIN_OBJ`: Defines the object files for the library and the main program.
- `NP` & `OMP_THREADS`: Defines the number of processes (MPI) and threads (openMP) to use for the parallel version of the program. The default values are set to 4 and 2, respectively, but they can be overridden by setting them when compiling in the command line (e.g., `make run_parallel NP=7 OMP_THREADS=3`).

In order to encompass both the serial and parallel versions of the program, a check is performed on the `make` command entered on the command-line for the word `parallel`. If found, the user's system is searched to ensure the environment has the necessary MPI and OpenMP libraries installed. If not, an error message is printed and the program exits. If they are installed, the

compiler variable `FC` is overwritten and the MPI and OpenMP flags are added to the `FFLAGS` variable.

[] ifneq (,($findstringparallel$,(MAKECMDGOALS))) # MPI Compiler Wrapper Verification MPI_BIN := $(shellcommand-vmpif902 > /dev/null)ifeq((strip (MPI_BIN)),)(error "MPI Compiler is required but 'mpif90' was not found! Please install it (e.g. 'sudo apt install openmpi-bin')") endif # Overwrite Compiler for MPI parallelization flag linking FC = mpif90 ...

Instead of writing a rule for every single `.f90` file, a pattern rule (with `%`) is used.

[] (OBJ_DIR)/%.o : lib/%.f90@$mkdir-p$(OBJ_DIR) (FC)(FFLAGS) -J(OBJ_DIR)$-I$(OBJ_DIR) -c $< -o$@

- It says: "To make any `.o` file in the object directory (`bin/`) from a matching `.f90` file in `lib/`, run this command."
- `-c $<`: Compiles the "first dependency" (`$<`, which is the `.f90` file) without linking (because that's done in the `$(EXE)` target).
- `-J$(OBJ_DIR) -I$(OBJ_DIR)`: Tells Fortran to put module `.mod` files in the `bin/` folder.

The following is an example of the `all` target (serial version of the program) with its executable structure, which we then follow up with the explicit dependencies to create the `.o` files for the library modules and the main program

[] all: (EXE)

(EXE) : (LIB_OBJ) ($MAIN_OBJ$)@$mkdir-p$(OBJ_DIR) (FC)(FFLAGS) -o @(LIB_OBJ) ($MAIN_OBJ$)

(OBJ_DIR)/$energy.o$: $lib/energy.f$90\(OBJ_DIR)/parameters.o ...

(OBJ_DIR)/$main_serial.o$: $main_serial.f$90(LIB_OBJ) @mkdir -p (OBJ_DIR)(FC) ($FFLAGS$)$-J$(OBJ_DIR) -I(OBJ_DIR)$-c<$ -o @ ...

- `all`: This is the default target that needs `$(EXE)` to exist.
- `$(EXE)`: To build the final executable, it needs all the .o files from the `../bin` folder, which it makes sure exists (`mkdir -p`), and then links them all together using `gfortran` or `mpif90` to output (`-o $@`) the final program.
  - **Note**: The `@` symbol is used 2 different ways here:
    1. Before the `mkdir` command to suppress being printed to the terminal.
    2. As an automatic variable representing the target name (`$(EXE)` in this case).

Last, we declare the shortcut commands as `.PHONY` targets to prevent conflicts with files that may have the same name as a target.

[] .PHONY: all clean run figures pipeline

run: all @mkdir -p ../results (EXE)

figures: python lib/plot_results.py

clean: rm -rf (OBJ_DIR)/$*.o$(OBJ_DIR)/*.mod (EXE)

pipeline: ($MAKE$)run(MAKE) figures

- This enables the following compile-time shortcuts:
  - `make run`: Automatically builds the code (`all`) and then executes it.

- `make figures`: Envokes python to generate the plots from the `../results` folder.
- `make clean`: Acts like a reset button, deleting all compiled files to start fresh.
- `make pipeline`: A neat wrapper we built to run the code and generate the python plots back-to-back.

0.5 Shell Scripting

Shell scripting was used to make our runs compatible with the CERQT2 cluster environment and to automate benchmarking. The scripts initialize the runtime environment, load the required modules, and set variables such as `OMP_NUM_THREADS`, `NSLOTS`, and `MPLBACKEND=Agg` for stable non-interactive execution. They were also used to automate parameter sweeps, build explicit trajectory manifests for the observables analysis, and collect timing data into summary files for MPI and OpenMP tests.

A qsub script `run_collab.qsub` was also written during the testing of the program to submit an individual portion of the simulation to the HPC cluster. It involved loading the appropriate modules for that queue, selecting a custom makefile (separate from the main program's Makefile), indentifying the correct commandline arguments to pass at compile time, and submit the mpirun command.

1 Initial Configuration Generation

1.1 Internal-Coordinate Builder

The `initial_conf` module is responsible for generating the starting conformation of the polyethylene chain. The builder uses a recursive internal-coordinate approach: each successive carbon atom is placed by specifying a fixed C–C bond length ($d = 1.54\,\text{Å}$), a fixed C–C–C bond angle ($\theta = 114°$), and a selectable dihedral angle ϕ. A hard-sphere overlap check with threshold $r_{\min} = 2.0\,\text{Å}$ is performed after each placement to avoid unphysical self-intersections during chain construction.

```
! Place carbon i based on the last two accepted positions
do i = 4, n_carbons
  ! Bond vector from i-2 to i-1 (normalised)
  b1 = coords(i-1,:) - coords(i-2,:)
  b1 = b1 / vnorm(b1)

  ! Normal to the C(i-2)-C(i-1)-C(i) plane
  b0 = coords(i-2,:) - coords(i-3,:)
  n_vec = cross(b0, b1)
  if (vnorm(n_vec) > 1.0d-8) n_vec = n_vec / vnorm(n_vec)

  ! Proposed new position using dihedral phi
  d_vec = bond_length * (                              &
      -cos_theta * b1                                  &
      + sin_theta * (cos_phi * n_vec                   &
                   + sin_phi * cross(b1, n_vec)))

  candidate = coords(i-1,:) + d_vec

  ! Overlap check against all previously placed atoms
  overlap = .false.
  do j = 1, i-1
    if (vnorm(candidate - coords(j,:)) < r_min) then
      overlap = .true.
```

```
      exit
    end if
  end do
  if (.not. overlap) coords(i,:) = candidate
end do
```

Listing 1: Core internal-coordinate placement loop in `initial_conf.f90`.

When explicit hydrogens are requested (`explicit_h = .true.`), two hydrogen atoms are appended for each internal CH_2 group and one for each terminal CH_3 group, giving $N_{\text{atoms}} = 3N_C + 2$ total atoms for a chain of N_C carbons. The hydrogen placement is based on a local tetrahedral frame: instead of placing the H atoms coplanar with the C–C–C backbone (chemically inconsistent), they are positioned out of the backbone plane at the correct tetrahedral angle of $\approx 109.5°$.

```
! For each internal carbon i (not first or last):
! Build local orthonormal frame {b_fwd, perp, out_of_plane}
b_fwd = coords(i+1,:) - coords(i-1,:)
b_fwd = b_fwd / vnorm(b_fwd)

! Perpendicular to backbone, in the C(i-1)-C(i)-C(i+1) plane
b_back = coords(i-1,:) - coords(i,:)
b_back = b_back / vnorm(b_back)
perp   = b_back - sum(b_back*b_fwd)*b_fwd
if (vnorm(perp) > 1.0d-8) perp = perp / vnorm(perp)

! Out-of-plane direction (cross product)
out_of_plane = cross(b_fwd, perp)
out_of_plane = out_of_plane / vnorm(out_of_plane)

! Place H1 and H2 symmetrically out of plane (tetrahedral geometry)
! cos(109.5 deg) ~ -1/3  =>  sin(109.5 deg) ~ sqrt(8/9)
h1 = coords(i,:) + r_CH * ( (-1.0d0/3.0d0)*(-perp)  &
     + sqrt(8.0d0/9.0d0) * 0.5d0 * ( out_of_plane + b_fwd) )
h2 = coords(i,:) + r_CH * ( (-1.0d0/3.0d0)*(-perp)  &
     + sqrt(8.0d0/9.0d0) * 0.5d0 * (-out_of_plane + b_fwd) )
```

Listing 2: Out-of-plane hydrogen placement for internal CH_2 groups.

1.2 Configuration Modes

Five distinct initial configuration modes have been implemented, each designed to probe a different region of the polymer's conformational space:

Configurations 1 and 5 span the near-trans region of phase space, whereas configurations 2 and 3 provide disordered starting points. Configuration 4 was added to test helical initial geometries. This variety is relevant when the simulation is tested at different temperatures: some initial states may lie closer to the equilibrated ensemble than others, reducing the effective burn-in time.

2 Input/Output Module

2.1 Parameter Parsing from `input.dat`

All simulation parameters are read from a plain-text file `input.dat` through the `io` module. The parser applies default values for any missing keyword, strips inline comments (lines beginning

Table 1: Summary of implemented initial configuration types.

conf_type	Name	Description
1	All-trans (planar)	All dihedrals set to $\phi = 0°$; fully extended zigzag chain. Maximum end-to-end distance.
2	Random	Each dihedral drawn uniformly from $[-\pi, \pi]$; maximally disordered starting state.
3	RIS-like discrete	Dihedrals assigned with probabilities reflecting the Rotational Isomeric State model (trans, gauche$^+$, gauche$^-$).
4	Spring / helix	Progressive dihedral increment per bond, producing a helical backbone (proposed by ai-murphy).
5	Perturbed trans	Small random perturbation applied to the all-trans geometry; slight out-of-plane deviation while remaining close to the global minimum.

with `#`), and performs basic range validation. The following parameters are controlled from the input file:

Table 2: Simulation parameters read from `input.dat`.

Parameter	Default	Description
`n_carbons`	10	Number of backbone carbon atoms
`n_steps`	1,000,000	Number of MC production steps
`temperature`	300.0	Simulation temperature (K)
`conf_type`	1	Initial configuration mode (see Table 1)
`explicit_h`	.false.	Enable all-atom (explicit hydrogen) mode
`rng_seed`	42	Random number generator seed
`print_interval`	1,000	Steps between output writes
`max_delta`	0.5	Maximum dihedral rotation per MC step (rad)

Output is written in XYZ format: trajectories to `traj_`*`label`*`.xyz`, energies to `energy_`*`label`*`.dat`, and observables to `observables_`*`label`*`.dat`, where the label encodes the key simulation parameters (`n_carbons`, `conf_type`, `n_steps`, `temperature`) for unambiguous identification.

3 Energy Evaluation and Monte Carlo Engine

3.1 Energy Calculations

The energy evaluation layer is implemented through two complementary modules, `energy.f90` and `energy_all_atoms.f90`. Together, they provide the total-energy routines and the ΔE calculations required by the Metropolis acceptance/rejection step during Monte Carlo sampling. The split is deliberate: the united-atom (UA) pathway focuses on high-throughput conformational sampling, whereas the all-atom (AA) pathway extends the physical model to include explicit hydrogens and a more detailed topology.

For the united-atom model, the priority was computational efficiency. Because the torsional and non-bonded energy terms are evaluated continuously inside the Monte Carlo loop, even modest overhead in the energy routine would be amplified over millions of steps. The UA implementation therefore avoids unnecessarily expensive transcendental operations wherever possible.

A key optimization concerns the torsional term. Direct evaluation of a dihedral angle typically

requires `acos` and sometimes `atan2`, both of which are relatively expensive. To bypass this, the code computes $\cos\phi$ directly from the geometry using vector cross products and dot products of the plane normals defined by four successive backbone atoms. In practice, this replaces repeated angle reconstruction by a cheaper purely algebraic calculation. Guard clauses are also included to handle nearly collinear geometries safely by defaulting to a trans-like value when the normal vectors become numerically ill-defined.

The united-atom torsional potential follows the standard TraPPE-UA form,

$$U(\phi) = c_1(1 + \cos\phi) + c_2(1 - \cos 2\phi) + c_3(1 + \cos 3\phi).$$

Introducing $y = \cos\phi$ and using the trigonometric identities $\cos 2\phi = 2y^2 - 1$ and $\cos 3\phi = 4y^3 - 3y$, this expression can be rewritten as a cubic polynomial in y:

$$U(y) = (c_1 + 2c_2 + c_3) + (c_1 - 3c_3)y - 2c_2y^2 + 4c_3y^3.$$

This transformation removes transcendental function calls from the inner Monte Carlo loop and reduces the torsional evaluation to a short sequence of multiplications and additions.

The transition to explicit hydrogens required a more detailed description of the chain and therefore a separate all-atom treatment. For this purpose, the AA implementation adopts an OPLS-AA description of polyethylene. A naive implementation would be prohibitively expensive, because rotating a single C–C bond induces several distinct dihedral contributions across that bond. Evaluating all such contributions explicitly at every trial move would make the torsional part of the simulation the dominant bottleneck.

To avoid this, the AA implementation uses an effective backbone potential. Rather than evaluating every C–C–C–H and H–C–C–H contribution independently, their combined effect is absorbed into a single effective torsional expression acting only on the carbon backbone. This preserves the intended force-field behaviour while greatly reducing the cost of each Monte Carlo step. The same polynomial-expansion strategy used in the UA model is then reused with a different parameter set, so the all-atom torsional energy is also evaluated as a compact polynomial in $\cos\phi$.

The explicit-hydrogen model also requires a topology-aware treatment of non-bonded interactions. In molecular mechanics, Lennard-Jones terms are not evaluated indiscriminately between all pairs of atoms, because short-range bonded interactions are already represented elsewhere in the force field. For this reason, the code initializes a topology map that assigns each hydrogen to its parent carbon and constructs a boolean exclusion matrix. This matrix is then queried during energy evaluation to skip atom pairs that should not contribute to the non-bonded Lennard-Jones energy.

A second important optimization concerns the ΔE calculation during trial moves. Recomputing the full chain energy after every pivot would be unnecessarily expensive, especially in the all-atom model. Instead, the code identifies which atoms have actually moved and splits the system into two dynamic lists: a fixed set and a moved set. Only cross-interactions between these two sets can change during a rigid-body pivot. The ΔE routine therefore restricts the Lennard-Jones calculation to exactly those interactions whose relative distances may have changed, while routing each valid pair to the appropriate C–C, C–H, or H–H parameter set. This significantly reduces the amount of work per trial move and is one of the key reasons the AA model remains tractable.

3.2 Pivot Move and Monte Carlo Step

The conformational sampling mechanism used throughout the project is the pivot move. Rather than perturbing the whole chain diffusely, the algorithm selects one internal C–C bond and

rotates the entire tail of the polymer as a rigid body around that bond axis. This generates large collective rearrangements and allows the chain to explore configuration space much more efficiently than local coordinate displacements.

The rigid-body rotation is implemented with Rodrigues' rotation formula. If $\mathbf{v}$ is the position of an atom relative to the pivot point, $\mathbf{k}$ is the unit vector along the selected bond, and θ is the random rotation angle, the rotated vector is

$$\mathbf{v}_{\text{rot}} = \mathbf{v}\cos\theta + (\mathbf{k}\times\mathbf{v})\sin\theta + \mathbf{k}(\mathbf{k}\cdot\mathbf{v})(1-\cos\theta).$$

This construction preserves bond lengths and bond angles while applying a clean rigid-body rotation to the selected downstream segment.

In all-atom mode, the same geometric transformation must be applied not only to the carbon backbone but also to the hydrogens attached to every rotating carbon. Because these hydrogens represent an approximately tetrahedral sp^3 environment, any inconsistency between the carbon motion and the attached hydrogen motion would immediately distort the local geometry and generate unphysical configurations. The AA rotation routine therefore identifies the hydrogens attached to each moving carbon and applies the exact same pivot point, axis, and angle to them.

Each call to `mc_step` performs one trial move. First, the code draws a random internal bond index and a random rotation angle. Second, a trial configuration is generated by rotating the corresponding tail segment while leaving the current coordinates unchanged. Third, the change in energy is computed using the model-appropriate ΔE routine: a united-atom path for UA simulations and an all-atom path for AA simulations. Fourth, the trial move is accepted or rejected using the Metropolis criterion.

The Metropolis rule is standard. If the move lowers the total energy, it is accepted unconditionally. Otherwise, it is accepted with probability

$$P_{\text{acc}} = \exp(-\beta\Delta E), \qquad \beta = \frac{1}{k_B T}.$$

If accepted, the trial coordinates replace the current coordinates and the total, Lennard-Jones, and torsional energies are updated in place. If rejected, the trial geometry is discarded and the original state is retained. Repeating this procedure generates a Markov chain whose stationary distribution is the desired Boltzmann distribution.

3.3 Geweke Convergence Detection

Detecting equilibration in a Markov chain is non-trivial because successive configurations are strongly autocorrelated. Earlier versions of the workflow relied on a fixed burn-in length and a posterior post-processing check, but this was often too rigid and could either discard too much data or fail to detect persistent transients. To make equilibration detection adaptive, the serial Monte Carlo engine was extended with an in-simulation Geweke diagnostic.

The Geweke test compares the mean of an early window of the trajectory with the mean of a late window of the same trajectory. In our implementation, the monitored quantity is the total energy. If the chain has equilibrated, both windows should be consistent with the same stationary distribution. The resulting z-score is

$$z = \frac{\bar{x}_A - \bar{x}_B}{\sqrt{\text{SE}_A^2 + \text{SE}_B^2}},$$

where $\bar{x}_A$ and $\bar{x}_B$ are the mean energies in the early and late windows, and SE_A and SE_B are their standard errors.

A crucial point is that the standard errors cannot be estimated from the raw sample variance as if the measurements were independent. Monte Carlo trajectories are correlated, so such a treatment would underestimate the uncertainty and produce false positives. To account for this, the code uses batch means. The window is partitioned into contiguous blocks, the mean of each block is computed, and the variance of those block means is used as an estimator of the uncertainty in the presence of autocorrelation.

The implementation uses an accumulated buffer of $N = 300$ energy samples. The early window contains the first $f_A = 10\%$ of the buffer and the late window contains the last $f_B = 50\%$. With these values, the two windows contain $n_A = 30$ and $n_B = 150$ samples respectively. The block decomposition then produces several batch means for each window, from which the corresponding standard errors are estimated.

Under the null hypothesis of stationarity, the Geweke statistic follows an approximately standard normal distribution. Equilibrium is declared when

$$|z| < z_{\text{crit}} = 1.96$$

at the 95% confidence level on three consecutive evaluations. Requiring multiple consecutive passes prevents the algorithm from mistaking a temporary plateau for genuine equilibration. The diagnostic is evaluated periodically once the rolling buffer is full, so the simulation can stop the equilibration phase as soon as a statistically stable regime has been reached.

3.4 Main Program Workflow Notes

The serial driver was updated so that equilibration is no longer treated as a hard-coded number of initial steps. Instead, the program reads the simulation parameters from `input.dat`, builds the initial coordinates, and enters a dedicated equilibration phase that continues until the Geweke criterion is satisfied.

If a previously equilibrated geometry is already available, it can be loaded directly to bypass the initial search. Otherwise, the simulation starts from the generated initial chain and repeatedly performs Monte Carlo steps while monitoring the total energy and structural observables. This turns equilibration into a data-driven phase of the workflow rather than an externally imposed burn-in guess.

The main equilibration loop is therefore a `do while (.not. is_equilibrated)` structure. At each step, the code updates the temperature-dependent parameter β, performs one Monte Carlo trial move, and increments acceptance statistics when appropriate. Although the code retains a temperature schedule of the form

$$T = T_{\text{ini}} - \Delta T\,(\texttt{istep} - 1),$$

the project simulations themselves are run at fixed temperature, so this acts mainly as a retained hook for possible future simulated-annealing tests.

Observables and trajectory information are written only at `print_interval` checkpoints rather than every single Monte Carlo step. At those checkpoints, the code records the total energy, Lennard-Jones energy, torsional energy, radius of gyration, end-to-end distance, and other phase-specific outputs required later by the plotting scripts.

Once the Geweke criterion has been satisfied for the required number of consecutive checks, the equilibration phase ends. The equilibrated geometry is saved and used as the starting point for the production phase. The production run then proceeds with its own step counter and output files, ensuring that the equilibrated and production data remain cleanly separated both scientifically and in the downstream post-processing pipeline.

4 Serial Driver, Results and Post-Processing

4.1 Two-Phase Simulation Structure

The updated serial workflow is built around a two-phase execution model: an adaptive equilibration stage followed by a fixed-length production stage. This is implemented in `main_serial_equil.f90`, where the program first reads `input.dat`, constructs the initial polymer geometry, and then decides whether equilibration should be performed from scratch or bypassed through a previously saved equilibrated configuration.

During Phase 1, the simulation performs Monte Carlo trial moves until the Geweke convergence diagnostic declares that the energy time series has reached a stationary regime. This replaces the old practice of assigning an arbitrary burn-in length in advance. Once equilibration is detected, the corresponding geometry is written to disk and becomes the starting point for Phase 2.

Phase 2 is the production run proper. Here, the serial driver resets the step counter and performs the requested number of Monte Carlo steps starting from the equilibrated geometry. In the serial reference case used throughout the report, this corresponds to a 10,000,000-step production simulation for a 500-carbon polyethylene chain with explicit hydrogens at 300 K.

The driver records its scientific output periodically through the `print_interval` mechanism. At each output checkpoint, the code writes the total, Lennard-Jones, and torsional energies together with the main structural observables, especially the radius of gyration R_g and the end-to-end distance R_{ee}. Torsion-angle data and trajectory snapshots are also written so that the structural evolution can be visualised afterwards.

An annealing schedule is still retained in the serial code in the form of a generic $T_{\mathrm{ini}} \rightarrow T_{\mathrm{fin}}$ update. For the simulations discussed here, however, the temperature is held fixed at 300 K, so this block acts mainly as a configurable extension point rather than a physically active part of the production protocol.

As a performance reference, the serial code was run on a single core of an Intel Core i7-11800H processor. In the benchmark comparison used in the report, one full serial production run required 8 hours, 18 minutes, and 31.91 seconds. This timing serves as the baseline against which the parallel implementations were later compared.

4.2 Automated Equilibration Detection

The serial results are post-processed through `plot_results.py`. This script begins by parsing `input.dat` so it can detect whether the run used explicit hydrogens and therefore decide which theoretical torsional potential should be overlaid in the final histogram plots. It then reads the separate equilibration and production outputs and stitches them together into a single coherent visual timeline.

For the energy and observable plots, the script concatenates the two phases and marks the transition with a vertical dashed line. This makes the time-demarcation between equilibration and production explicit and allows the reader to assess whether the dynamics truly become stationary after the detected equilibration point. By contrast, the torsional distribution is generated using only the production portion, since that is the only phase intended to represent equilibrium sampling.

The plotting layer is organized around three main functions: `plot_energies`, `plot_observables`, and `plot_torsions`. The first plots the total, Lennard-Jones, and torsional energies as functions of Monte Carlo step. The second plots R_g and R_{ee}. The third constructs the torsional histogram and superposes the corresponding theoretical torsional potential using the appropriate coefficient set for either the TraPPE-UA or OPLS-AA effective model.

Before the runtime Geweke detector was introduced, a post-processing equilibration analysis was also used as a diagnostic layer. In that approach, the Python workflow estimated the integrated autocorrelation time τ_{int} from the energy series by computing the autocorrelation function through an FFT-based convolution. This provides a fast way to estimate correlation lengths and effective sample sizes even for long trajectories.

That legacy Python analysis remains useful as a cross-check because it gives a purely post hoc view of correlation decay, burn-in length, and statistical inefficiency. However, it was not always robust enough to control the simulation by itself. In practice, long correlated plateaus could distort the variance estimates and lead to pathological truncations, including cases where the inferred production segment became unreasonably short. This limitation was one of the main reasons for moving equilibration detection into the Fortran runtime loop via the Geweke criterion.

The final post-processing pipeline therefore combines two roles. The Fortran code performs the actual online equilibration detection and separates the simulation cleanly into equilibration and production, and the Python code acts as the analysis and visualization layer that verifies, displays, and interprets the resulting time series and distributions.

4.3 Serial Version Simulation Results

The serial reference simulation considered here consists of an equilibration run followed by a 10,000,000-step production run for a 500-carbon polyethylene chain with explicit hydrogens at 300 K. The initial geometry starts from a highly extended chain with a fixed dihedral construction, after which the Monte Carlo engine samples torsional rotations across the full $-\pi$ to π range.

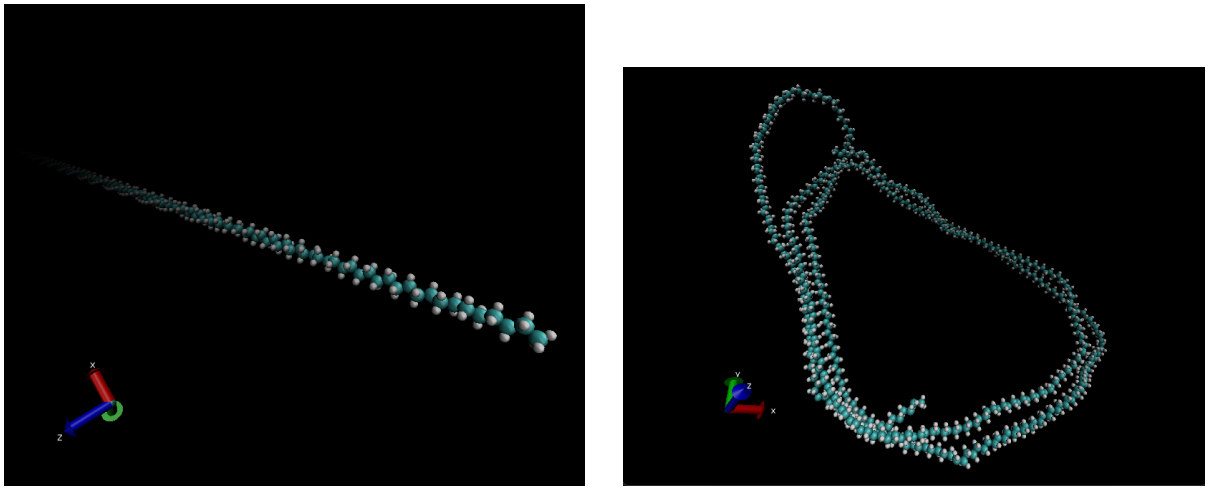

Figure 1: Initial and final configurations for the serial equilibration/production test.

The initial and final structures show the large-scale conformational reorganization produced by the Monte Carlo dynamics. The simulation starts from an almost fully extended configuration and evolves toward a much more compact and disordered polymer conformation, consistent with a random-coil-like state after equilibration.

The observables plot gives the clearest structural signature of this relaxation. Initially, both R_g and R_{ee} are very large because the chain is nearly stretched out. As trial rotations accumulate, the chain rapidly folds and collapses. In the data used for the report, the Geweke criterion identifies the transition to equilibrium at roughly 3.9 million steps, after which both observables fluctuate within comparatively stable bounds characteristic of the production regime.

The energy evolution shows an analogous story. At early times, the Lennard-Jones contribution

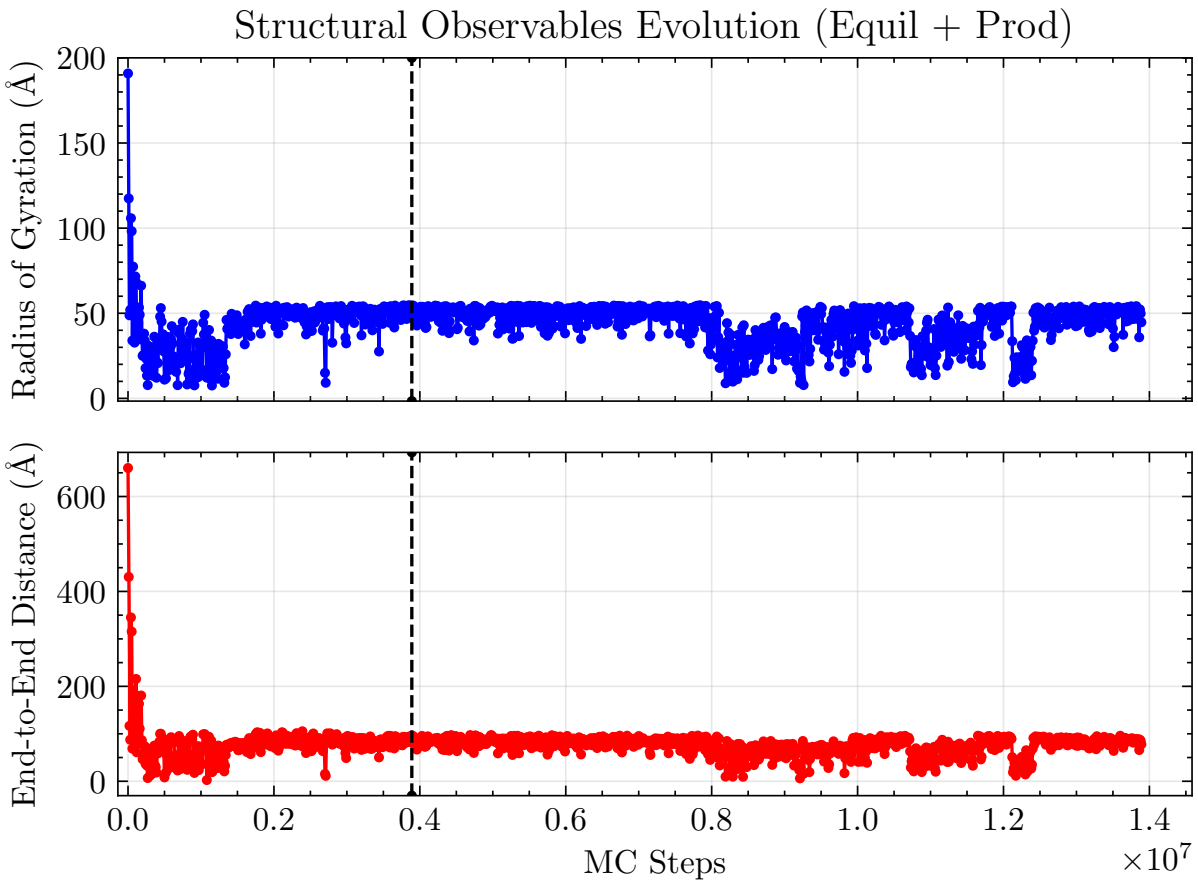

Figure 2: Evolution of the radius of gyration and end-to-end distance across equilibration and production.

drops sharply as the artificially regular starting configuration relaxes and resolves its initial steric unfavourability. After the equilibration boundary, the total energy fluctuates around a much more stable mean value, indicating that the chain has reached a stationary conformational ensemble rather than continuing to drift systematically.

The production-only torsional histogram reveals the angular regions most frequently explored once the chain has equilibrated. In our runs, the distribution is strongly concentrated near values close to 0 rad. This differs from what one might expect from the raw standalone torsional potential alone, but it is consistent with a total-energy landscape in which compact states are heavily rewarded by non-bonded packing.

4.4 Interpretation

A useful way to interpret the serial results is to view the simulation as a balance between energetic stabilization and configurational entropy. At 300 K, the polymer is not searching only for a single minimum-energy geometry. Instead, it samples according to the Boltzmann distribution, so many structurally distinct compact states can be favoured collectively over one perfectly extended low-energy idealized state. This is why the chain coils rather than remaining elongated.

The torsional histogram also should not be interpreted as a direct copy of the bare analytical

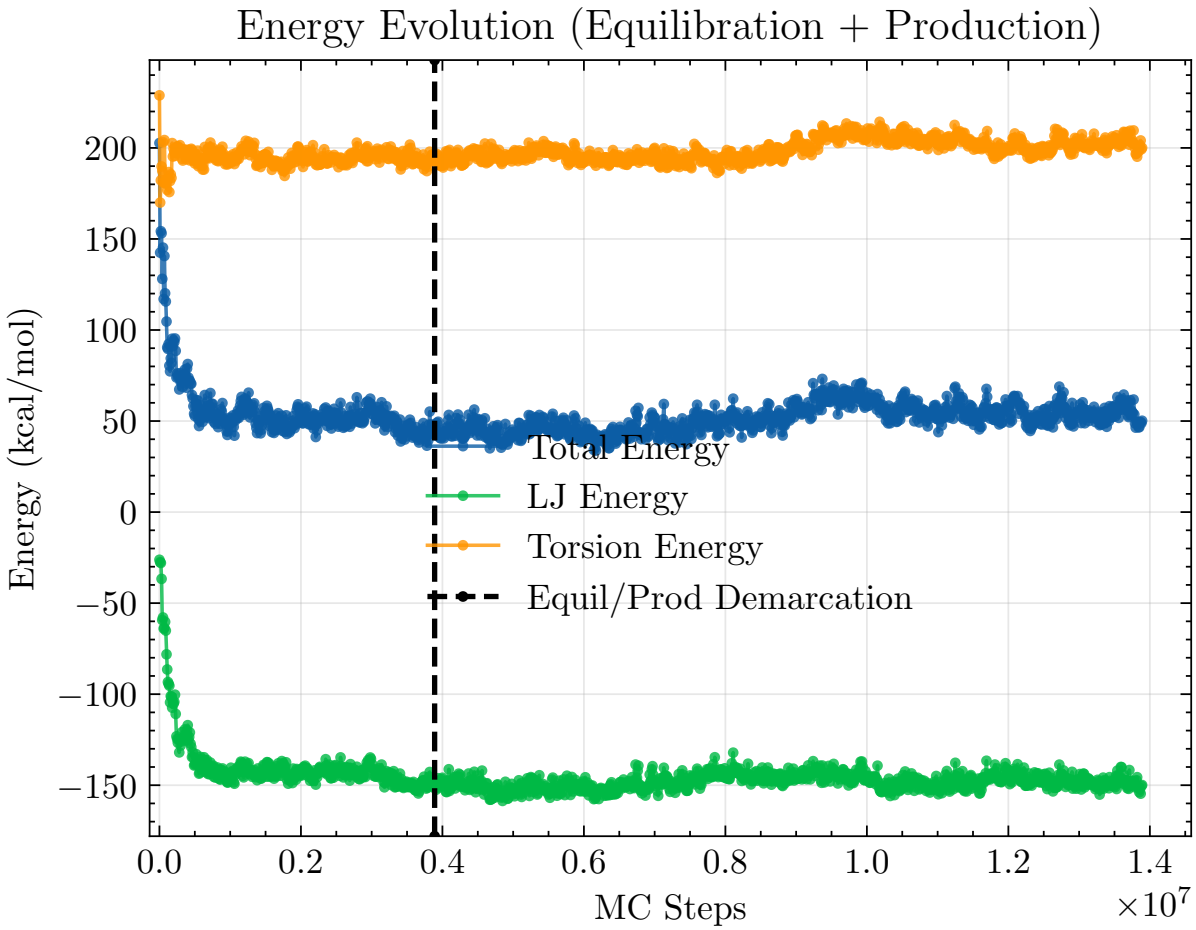

Figure 3: Energy evolution across equilibration and production.

torsional potential. The Monte Carlo acceptance criterion is based on the *total* energy,

$$E_{\text{tot}} = E_{\text{LJ}} + E_{\text{tors}},$$

not on the torsional term in isolation. A conformation with a locally unfavourable torsion can still be accepted if that penalty is more than compensated by favourable packing elsewhere in the chain.

In the present all-atom implementation, there is also a specific topology-related reason why the torsional distribution can become biased toward compact cis-like states. Short-range Lennard-Jones interactions between atoms mapped as separated by fewer than four backbone bonds are excluded in order to avoid double-counting interactions already encoded in bonded terms. This is standard in force-field implementations, but when explicit hydrogens are present it can suppress a strong steric repulsion that would otherwise penalize very tight local curling across the bond.

At the same time, longer-range contacts such as 1–5, 1–6, and higher hydrogen-hydrogen interactions are still retained. Once the chain folds into a dense globule, many of these pairs can sit near the attractive region of the Lennard-Jones well simultaneously. The net result is that compact structures can gain a substantial favourable E_{LJ} contribution, enough to outweigh the local torsional penalty associated with angles near 0 rad.

For this reason, the observed torsional distribution should be read as a property of the implemented total-energy model rather than as a direct validation of the isolated OPLS-AA or

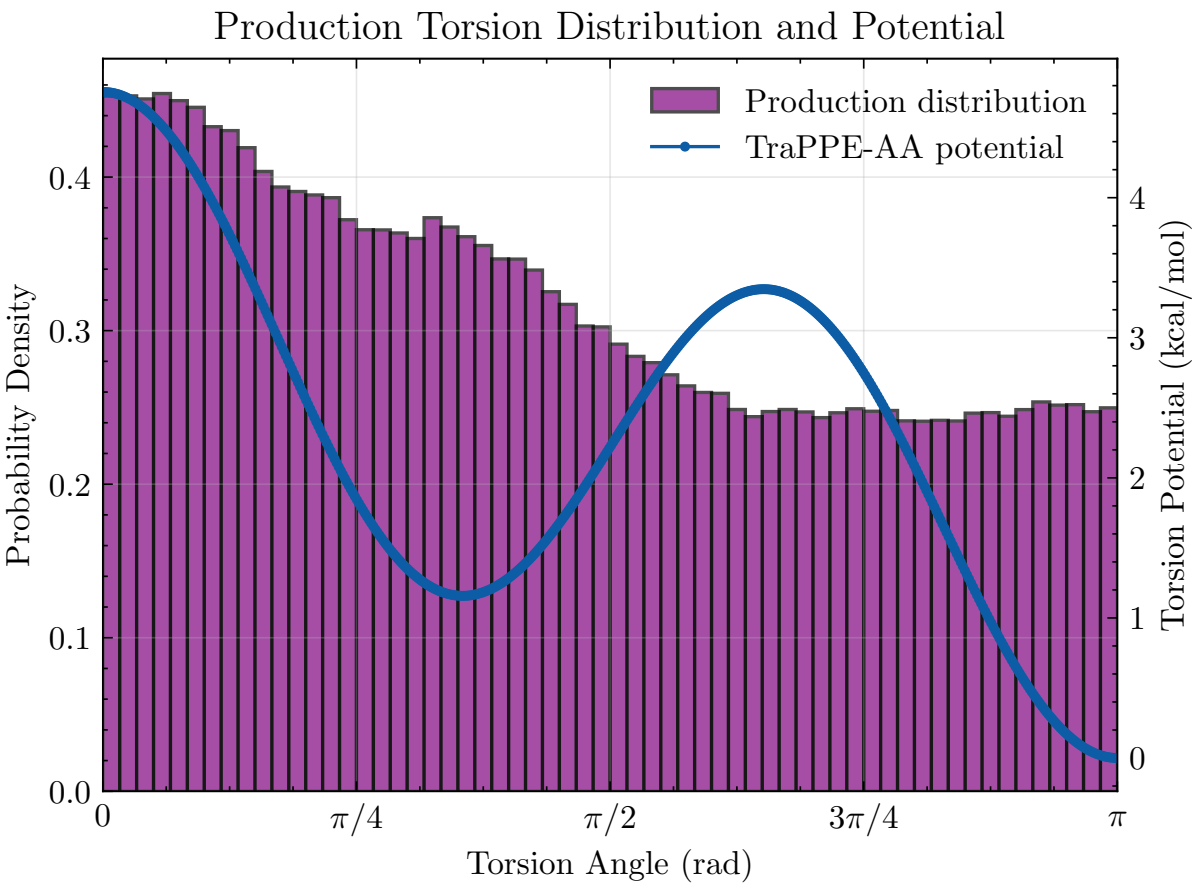

Figure 4: Production-only torsional distribution.

TraPPE torsional curve. A future refinement would be to replace absolute short-range exclusion by a more standard scaled treatment of selected short-range interactions, which would likely rebalance the competition between local steric repulsion and longer-range packing.

5 Cluster Compatibility

Portability to shared HPC infrastructure required two targeted modifications. First, the `Makefile` replaces the Fortran 2008 `open(newunit=u)` syntax with explicit unit-number declarations compatible with Fortran 90 compilers available on the CERQT2 cluster. Second, a submission script `1.run.sh` was created to load the appropriate GCC and OpenMPI modules before compilation and execution. The `parameters.f90` file was adapted to restore the `#ifdef` preprocessor block controlling the `explicit_h` compilation flag, and a comment was added to the `Makefile` explaining why the `-I$(MPI_INC_DIR)` flag is conditionally omitted in certain build configurations.

6 MPI Parallelization: Independent Replicas

6.1 Justification

The MPI parallelization implemented here is based on independent replica parallelism as a first step, while the energy evaluation and the Monte Carlo kernel can be parallelized subsequently (see the Energy Evaluation and Monte Carlo sections). First, independent replicas do not require inter-process communication inside the Monte Carlo loop, which keeps synchronization overhead

very low. Second, running several initial configurations simultaneously makes it possible to assess how sensitive the final sampled ensemble is to the choice of starting geometry.

6.2 Implementation: `main_parallel_replicas.f90`

Starting from `main_serial.f90`, three MPI ranks are launched simultaneously. Each rank runs the identical Monte Carlo protocol (same thermodynamic parameters, same number of steps) but is assigned a distinct initial configuration and a perturbed random seed:

```
! MPI initialization
call MPI_Init(ierr)
call MPI_Comm_rank(MPI_COMM_WORLD, rank, ierr)
call MPI_Comm_size(MPI_COMM_WORLD, n_ranks, ierr)

! Assign initial configuration based on rank
select case (rank)
  case (0); conf_type = 1   ! all-trans (extended)
  case (1); conf_type = 4   ! helix / spring
  case (2); conf_type = 5   ! perturbed trans
end select

! Perturb the RNG seed so trajectories immediately diverge
rng_seed = rng_seed + rank * 104729

! Build the initial configuration and run the full MC protocol
call build_initial_conf(n_carbons, conf_type, explicit_h, coords,
   symbols)
call mc_run(...)

! Rank-labelled output so files do not overwrite each other
write(label, '(A,I1)') trim(base_label)//'_rank', rank
call write_trajectory(label, coords, symbols, n_atoms)
```

Listing 3: Rank-dependent configuration and seed assignment.

The three configurations chosen (`conf_type` 1, 4, and 5) cover the near-trans region from three distinct but close starting points: fully extended, helical, and slightly perturbed extended. This choice maximises the geometric diversity of the initial ensemble while keeping all replicas in a physically reasonable region of conformational space.

A serial counterpart, `main_serial_replicas.f90`, runs the same three configurations sequentially to provide a controlled performance reference. Both versions employ wall-clock timing: `MPI_Wtime` in the parallel driver and an equivalent system-clock call in the serial one, ensuring that the speedup ratios reported in Section 10 reflect actual elapsed time rather than CPU time.

6.3 Discarded Parallelization: XYZ Trajectory Writing

A second parallelization direction was explored: distributing the XYZ trajectory write step across ranks, so that each rank writes its own frame independently. The resulting wall-clock improvement was marginal (less than 2% of total runtime), confirming that trajectory I/O is not the computational bottleneck. The dominant cost lies in the energy evaluation and Metropolis acceptance/rejection logic, as expected from the $\mathcal{O}(N^2)$ Lennard-Jones loop. This branch was therefore discarded and the simpler rank-labelled sequential write was kept.

7 MPI Parallelization: Observable Post-Processing

7.1 Design and Implementation

Post-processing of the trajectory data has been parallelized in `main_parallel_observables.f90`. Rather than decomposing a single trajectory into contiguous frame blocks, the implementation operates at the file level: all trajectory files are first enumerated in a manifest, rank 0 reads this manifest, and the complete file list is broadcast to all MPI ranks. Each rank then processes a round-robin subset of trajectory files, independently computing the radius of gyration R_g, the end-to-end distance R_{ee}, and the torsional statistics for the configurations assigned to it.

This design was chosen because the available data are naturally distributed across multiple trajectory files, corresponding to different configuration types and simulation phases (`equil` and `prod`). A file-level distribution avoids repeated file seeks inside large XYZ trajectories, keeps the implementation simple, and preserves a fully embarrassingly parallel local workload. At the same time, it remains fully compatible with collective MPI communication for global aggregation.

A first collective step is performed through `MPI_Bcast`, which is used to distribute the manifest and the maximum number of torsional degrees of freedom to all ranks. After each rank has completed its local file subset, a mid-run checkpoint is evaluated by means of `MPI_Allreduce`. In this step, the partial frame counts and partial R_g sums are synchronised across all processes, so that the global partial mean can be computed before the final aggregation stage. This checkpoint was introduced as a lightweight example of active communication during the computation phase, beyond the final collection of results.

```
! Each rank processes a round-robin subset of trajectory files
do ifile = rank + 1, nfiles, num_procs
  call process_trajectory(trim(files(ifile)), max_phi, &
       local_file_count, local_frame_count, &
       local_sum_rg, local_sum_rg2, local_sum_ree, local_sum_ree2, &
       local_nphi, local_sum_phi, local_sum_phi2)
enddo

! Mid-run checkpoint: all ranks obtain the global partial mean of Rg
call MPI_Allreduce(local_frame_count, ckpt_frame_count, n_conf*n_phase
   , &
                   MPI_INTEGER, MPI_SUM, MPI_COMM_WORLD, ierr)
call MPI_Allreduce(local_sum_rg, ckpt_sum_rg, n_conf*n_phase, &
                   MPI_DOUBLE_PRECISION, MPI_SUM, MPI_COMM_WORLD, ierr
                      )

! Final aggregation on rank 0
call MPI_Reduce(local_sum_rg, global_sum_rg, n_conf*n_phase, &
                MPI_DOUBLE_PRECISION, MPI_SUM, 0, MPI_COMM_WORLD, ierr
                   )
```

Listing 4: Round-robin file distribution and MPI checkpoint in `main_parallel_observables.f90`.

The final global statistics are obtained on rank 0 through a series of `MPI_Reduce` operations, which combine the local sums, squared sums, frame counts, and torsional accumulators into global averages and standard deviations. The corresponding summary files (`summary_global_np1.dat`, `summary_global_np2.dat`, `summary_global_np4.dat`, and `summary_global_np8.dat`) show that the numerical results are identical for all tested values of n_p, confirming that the parallelization preserves the observable averages exactly.

The benchmark driver is invoked from the cluster submission script `3.run_parallel_observables_np.sh`,

which executes the same post-processing workflow for $n_p \in \{1, 2, 4, 8\}$, records both MPI and shell wall-clock times, and writes per-process summary files for subsequent comparison.

8 MPI Parallelization: Collaborative Equilibration

8.1 Orchestrator Initial Configuration Read and Broadcast

In the MPI version of the program, rank 0 acts as the central orchestrator. Its role is to read the initial configuration files, decide whether each conformation must still be equilibrated or can directly enter production, and distribute the corresponding work to the workers.

At startup, the master reads the input settings and the geometry required to initialise each polymer configuration. If an equilibrated geometry is already available for a given configuration, the program can bypass the equilibration stage and directly unlock the corresponding production replicas. Otherwise, the master marks that configuration as pending collaborative equilibration.

For configurations that still require equilibration, rank 0 assigns a group of workers to the same initial structure. These workers all begin from the same geometry, but each one uses a different random seed so their trajectories immediately diverge. The master therefore provides a common starting point while still allowing independent exploration of phase space.

This orchestrator stage also centralises the campaign-level file handling. Workers do not need to parse the full simulation manifest or determine which configurations are ready for production. Instead, they receive either the job descriptor or the coordinates required for the task they have been assigned, initialise their local arrays, and begin Monte Carlo propagation.

8.2 Equilibration Optimization

Rather than assigning a single worker to equilibrate each configuration type independently, a group of `workers_per_equil` MPI workers is assigned to each configuration type simultaneously. Each worker starts from the same initial geometry but with a different random seed, for example `rngseed = rank * 104729`, ensuring that trajectories immediately diverge and explore distinct regions of conformational space.

Every `sync_interval` Monte Carlo steps, a synchronisation protocol is executed. Each worker sends its current energy and R_g to the master, which selects the most representative configuration via a Boltzmann-weighted roulette wheel and broadcasts those coordinates back to the non-winning workers in the group. The winning worker continues its trajectory uninterrupted, preserving the best conformational path found so far.

The purpose of this strategy is to reduce the time required to reach a well-equilibrated low-energy configuration. Instead of waiting for one trajectory to escape a poor metastable region on its own, the algorithm lets several replicas search in parallel and periodically shares the best result across the group. In this way, all workers benefit from the progress made by the most successful replica.

8.3 Replica Selection: Boltzmann Roulette Wheel

At each synchronisation point, the master assigns a statistical weight to each replica i based on its total energy E_i:

$$w_i = \exp\left(-\frac{E_i - E_{\min}}{k_B T_{\text{virt}}}\right),$$

where $E_{\min}$ is the lowest energy in the current group and T_{virt} is a virtual temperature.

The virtual temperature is a purely algorithmic parameter with no physical meaning. It controls how aggressively low-energy replicas are favoured:

- $T_{\text{virt}} \to 0$: deterministic selection of the minimum-energy replica.
- $T_{\text{virt}} \to \infty$: uniform selection, ignoring energies.

A uniform random number then samples this distribution via roulette-wheel selection. The selected replica becomes the winner of that synchronisation round, and its coordinates are sent to the non-winning workers so they can resume the search from the most promising state found so far.

8.4 MPI Communication Protocol

The synchronisation uses only point-to-point MPI operations `MPI_Send` and `MPI_Recv` to avoid the need for temporary communicators or collective operations across worker subgroups. At each sync point, the master loops over all workers in the active equilibration group and performs the following exchanges.

Step	Direction	Tag	Content
1	worker → master	`TAG_SYNC_OBS`	E_{tot}, R_g^2
2	master → worker	`TAG_SYNC_CTRL`	control signal, `bestlocalidx`
3	worker → master	`TAG_EQUIL_COORDS`	full coordinate array
4	master → non-winners	`TAG_SYNC_COORDS`	winning coordinates

Table 3: Point-to-point synchronisation protocol used during collaborative equilibration.

Once the Geweke test declares equilibrium, the master saves the equilibrated geometry to `../results/equilibrated_cX.xyz` and unlocks the 10 production jobs for that configuration type in the task queue.

8.5 Metrics and Methodology

This section evaluates the performance of the equilibration optimization technique compared to the serial baseline. The system studied is configuration 1, a chain of 500 carbons at $T = 300$ K including hydrogens. Efficiency is evaluated based on the time to reach specific energy thresholds $E_{\text{tot}} = 110$, 100, and 90 kcal/mol.

The speedup is defined as

$$S_w = \frac{\text{Steps}_{\text{serial}}}{\text{Steps}_{\text{parallel},\, w}},$$

where w is the number of workers participating in the collaborative equilibration group. Efficiency is then defined as

$$\eta_w = \frac{S_w}{w}.$$

The table below summarises the number of MC steps required by different worker configurations to reach the target energy levels.

These results show that the method can exhibit superlinear speedup. This is not just because more workers are used, but because the search itself changes. A population of replicas explores several conformational regions at once, and once one replica finds a much better basin, that discovery is broadcast to the others at the next synchronisation point.

Threshold	Workers w	Steps Required	Speedup S_w	Efficiency S_w/w
110 kcal/mol	1	590,000	1.00	1.00
110 kcal/mol	2	70,000	8.43	4.21
110 kcal/mol	4	140,000	4.21	1.05
110 kcal/mol	8	50,000	11.80	1.47
100 kcal/mol	1	1,180,000	1.00	1.00
100 kcal/mol	2	350,000	3.37	1.68
100 kcal/mol	4	210,000	5.62	1.40
100 kcal/mol	8	50,000	23.60	2.95
90 kcal/mol	1	2,400,000	1.00	1.00
90 kcal/mol	2	1,780,000	1.35	0.67
90 kcal/mol	4	570,000	4.21	1.05
90 kcal/mol	8	160,000	15.00	1.87

Table 4: Monte Carlo steps required to reach selected energy thresholds during collaborative equilibration.

For this reason, the observed acceleration is partly computational and partly algorithmic. The collaborative protocol does not simply divide the same work among more processes; it improves the probability of finding favourable states earlier than a single serial trajectory would.

8.6 Ensemble Averaging and Star Topology

Once an equilibrated geometry has been found, the workflow switches from collaborative equilibration to large-scale production sampling. At this stage, rank 0 acts as a central dispatcher in a star-topology MPI scheme, while the remaining ranks behave as workers that repeatedly request new tasks from the master.

The master maintains a global queue of production jobs. Each job corresponds to an independent production replica, and workers are assigned new jobs dynamically as soon as they finish their current one. This avoids the load imbalance that would appear if all replicas were distributed statically at launch, since individual trajectories may still differ slightly in runtime.

The dispatcher listens using a tag-based communication pattern and can react to whichever worker becomes free first. In practice, this means that workers request work, receive a production assignment, run the simulation, write their outputs, and notify the master when they have finished. The master then either sends another job or terminates the worker once the queue has been exhausted.

This star-topology design turns the MPI layer into an ensemble averaging engine. Independent production replicas are generated as efficiently as possible, and the final observables are obtained by averaging over the outputs of all completed runs. In this phase, the objective is no longer to accelerate the equilibration of a single chain, but to maximize statistical throughput across the whole ensemble.

9 OpenMP Parallelization

9.1 `compute_total_energy`

The dominant cost in `compute_total_energy` is the $O(N^2)$ double loop over non-bonded Lennard-Jones pairs. Each iteration is independent — it reads a pair of coordinates, evaluates `ljpairenergy`, and adds to a running sum — which makes it a natural fit for OpenMP worksharing.

Figure 5: Convergence speedup analysis for collaborative equilibration.

Direction	Tag	Meaning
worker → master	`TAG_REQUEST_WORK`	worker requests a new task
master → worker	`TAG_DO_PROD`	start a production replica
master → worker	`TAG_WAIT`	remain idle temporarily
master → worker	`TAG_DIE`	terminate execution
worker → master	`TAG_PROD_DONE`	production replica finished

Table 5: Tag-based work distribution in the star-topology production phase.

The outer loop is distributed across threads with `!$omp parallel do`, with the inner index, the squared distance r^2, and the dihedral cosine `cosphi` declared `private` to prevent cross-thread interference. The global accumulators `elj` and `etors` are aggregated safely using a `reduction(+:...)` clause.

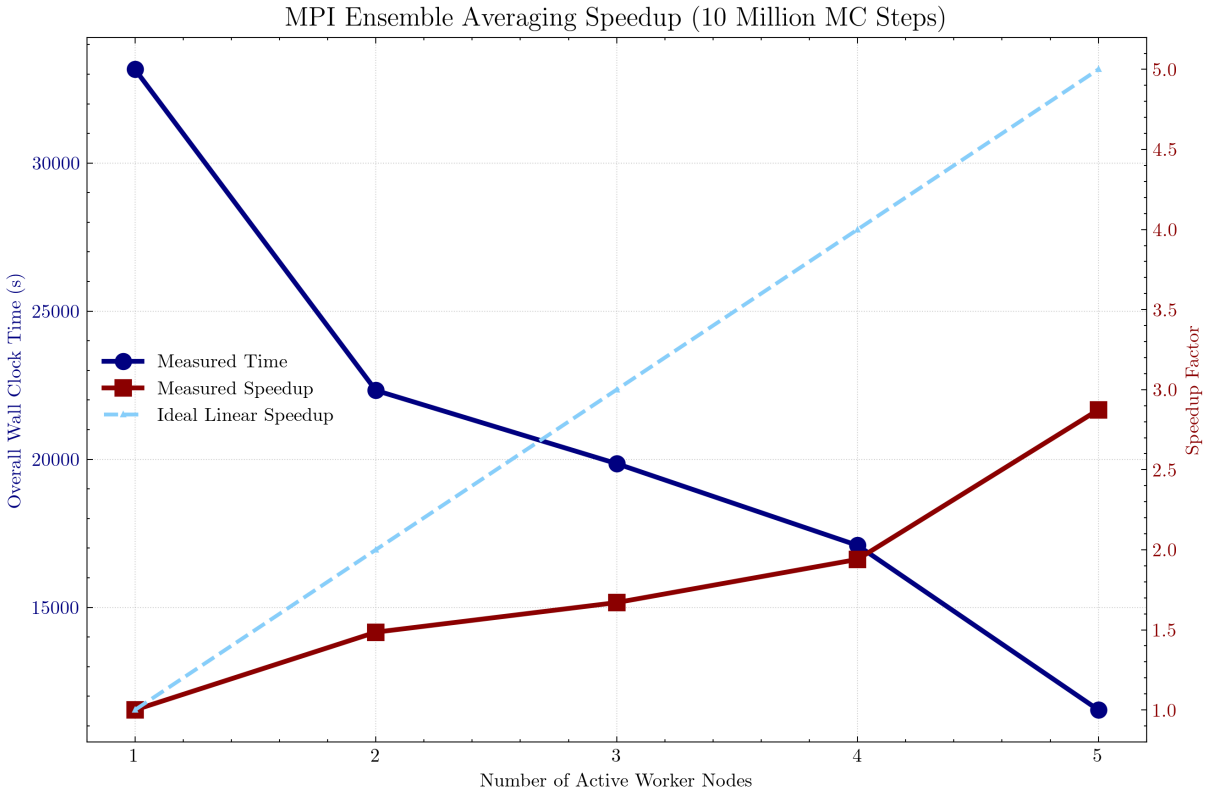

Figure 6: Timing and efficiency of the orchestrator-worker production workflow.

The two loops inside the subroutine have different iteration structures and were scheduled accordingly. The LJ pair loop has a triangular iteration space because the inner bound depends on the outer index, so `schedule(dynamic,10)` is used to prevent faster threads from sitting idle while slower ones finish larger chunks of work. The torsional loop over the carbon backbone is uniform in length and uses the default static scheduling, which has lower overhead when the workload is balanced.

Both parallel regions are wrapped in an `if(omp_totalenergy)` runtime guard, which is set from the input file. This allows the same binary to run in purely serial mode during development or lightweight tests without any code changes.

9.2 `delta_energy`

The `delta_energy` subroutine is the true bottleneck of the MC loop — it is called once per accepted or rejected move, millions of times per simulation. The key algorithmic feature here is the dynamic partitioning of atoms into `fixedlist` and `movedlist` at the start of each call.

Only cross-interactions between fixed and moved atoms can have changed during a pivot, so the LJ loop processes exclusively those pairs rather than the full $O(N^2)$ set. This drastically reduces the amount of work required for a local move and makes the subroutine much more sensitive to efficient parallelization.

The two nested loops over fixed and moved form a perfectly rectangular iteration space, which makes `collapse(2)` effective: it merges both loops into a single flat range of $n_{\text{fixed}} n_{\text{moved}}$ iterations and distributes them evenly across threads.

Since pivot moves near the ends of the chain move very few atoms, spawning threads for only

a small number of pairs would cost more than it saves. The parallel block is therefore gated by `if(omp_deltaenergy .and. nfixed*nmoved > 1000)`, ensuring that OpenMP is only activated when the workload genuinely justifies the threading overhead.

9.3 Benchmarking Methodology

To quantify the effect of thread count on both energy routines, a dedicated benchmark program `mainbenchtotal.f90` was written. It performs 10,000 back-to-back calls to `compute_total_energy` for a given chain size and reports the wall time via `MPI_Wtime`.

Two shell scripts automate the full benchmark sweeps. `4.runomptotaltest.sh` compiles and runs the benchmark binary across thread counts of 1, 2, 3, and 4 for chain sizes of 20, 50, 100, and 500 carbons, writing the results to a CSV file in the benchmark results folder.

`5.runompdeltatest.sh` follows the same structure but targets `delta_energy` by running full 1M-step MC simulations via `mainparallelreplicas.x` and extracting wall time from the CPU output files. Both scripts are written for the cluster queue and load the appropriate MPI module.

The results are plotted by `plotbenchmarks.py`, which reads both CSV files and produces per-chain-size scaling plots as PDF figures.

A note must be made regarding the `delta_energy` benchmark: because it is measured inside full Monte Carlo production runs, the timings are affected by randomness. Different random trajectories lead to different accepted moves and different moved/fixed partitions, so the workload is not bitwise identical between runs. For that reason, the `delta_energy` benchmark should be interpreted as a practical end-to-end timing measurement rather than as a perfectly controlled microbenchmark.

9.4 SIMD Exploration

An attempt was made to additionally exploit single-core SIMD vectorization on the `ljpairenergy` function using the `!$omp declare simd` directive. The idea was to generate a vectorized variant of the function callable from an `!$omp simd`-annotated inner loop, allowing each thread to process multiple atom pairs simultaneously using AVX registers.

In practice, activating this required structural changes to `delta_energy` that conflicted with the existing threading design. The `collapse(2)` clause and the inner-loop `cycle` on `isexcluded` are not naturally compatible with SIMD regions, and removing them in order to support vectorization degraded the thread load balancing that `collapse(2)` provides.

Given that OpenMP threading was the primary parallelization goal, the SIMD experiment was abandoned and the directive was removed from the code. It remains a direction worth revisiting, particularly if the pair loops are later refactored to use pre-filtered pair lists that eliminate the exclusion branch entirely.

10 Scaling Analysis and Discussion

10.1 Replica Parallelism: Speedup vs. System Size and Run Length

The independent-replica MPI strategy was benchmarked by varying both the polymer size and the production run length. In this approach, each MPI worker executes a different production replica, so the total wall time depends primarily on how much useful work can be distributed across workers before communication and orchestration overhead begin to matter.

The first scaling study varies the number of carbons while keeping the simulation length fixed. As the chain becomes larger, each replica contains more internal work, especially in the energy

calculations and observable evaluations, so the MPI orchestration overhead becomes less significant relative to the total runtime. For this reason, larger systems are expected to display better parallel efficiency than very small ones.

The second study varies the number of Monte Carlo steps for a fixed system size. Short runs do not amortize the startup, scheduling, and I/O costs very well, whereas longer runs allow each worker to spend a much greater fraction of time performing actual MC sampling. Consequently, speedup generally improves as the number of production steps increases.

In both cases, the speedup is not expected to be perfectly linear. The master node still performs administrative tasks such as initial file reading, dispatching jobs, receiving completion messages, and managing the production queue. This means that part of the total hardware budget is always reserved for orchestration rather than direct numerical work.

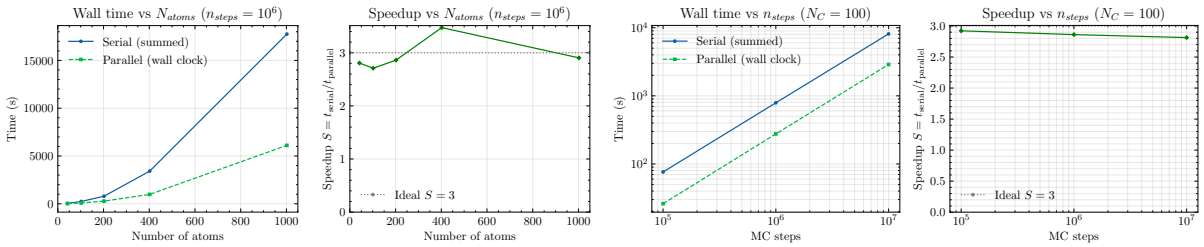

Figure 7: Speedup of independent-replica parallelization.

10.2 Observable Post-Processing: Strong Scaling

A second scaling study was carried out for the observable post-processing kernel, implemented in `main_parallel_observables`. Unlike the production simulations, this stage is a strong-scaling problem: the total dataset is fixed, and the number of MPI processes is increased to measure how efficiently the same workload can be completed.

Each process reads a subset of trajectory files, computes the radius of gyration, end-to-end distance, and torsional statistics, and accumulates local partial sums. At the end of the run, the global results are assembled through MPI reductions on rank 0, which writes the final averaged observables.

This workload is well suited to MPI because each trajectory file can be processed independently for most of the runtime. Communication is only required at the end, when the local statistics are combined into a single global dataset. As a result, the observable analysis behaves much closer to an embarrassingly parallel workload than the simulation phase itself.

The scaling is therefore expected to be strong for moderate process counts, with efficiency gradually decreasing once the number of workers becomes large compared to the number of files. At that point, the amount of useful work per process shrinks and the fixed costs of process launch, file I/O, and final reductions become more visible.

10.3 Discussion

One of the key observations from our performance analysis is that parallelization rarely translates into perfectly linear speedup. In an ideal case, doubling the number of cores would halve the runtime, but in practice communication overhead, process coordination, file handling, and hardware contention all reduce the efficiency of additional workers.

This effect is visible in both the independent-replica and post-processing benchmarks. For the MPI production workflow, the master must coordinate task dispatch and receive status messages

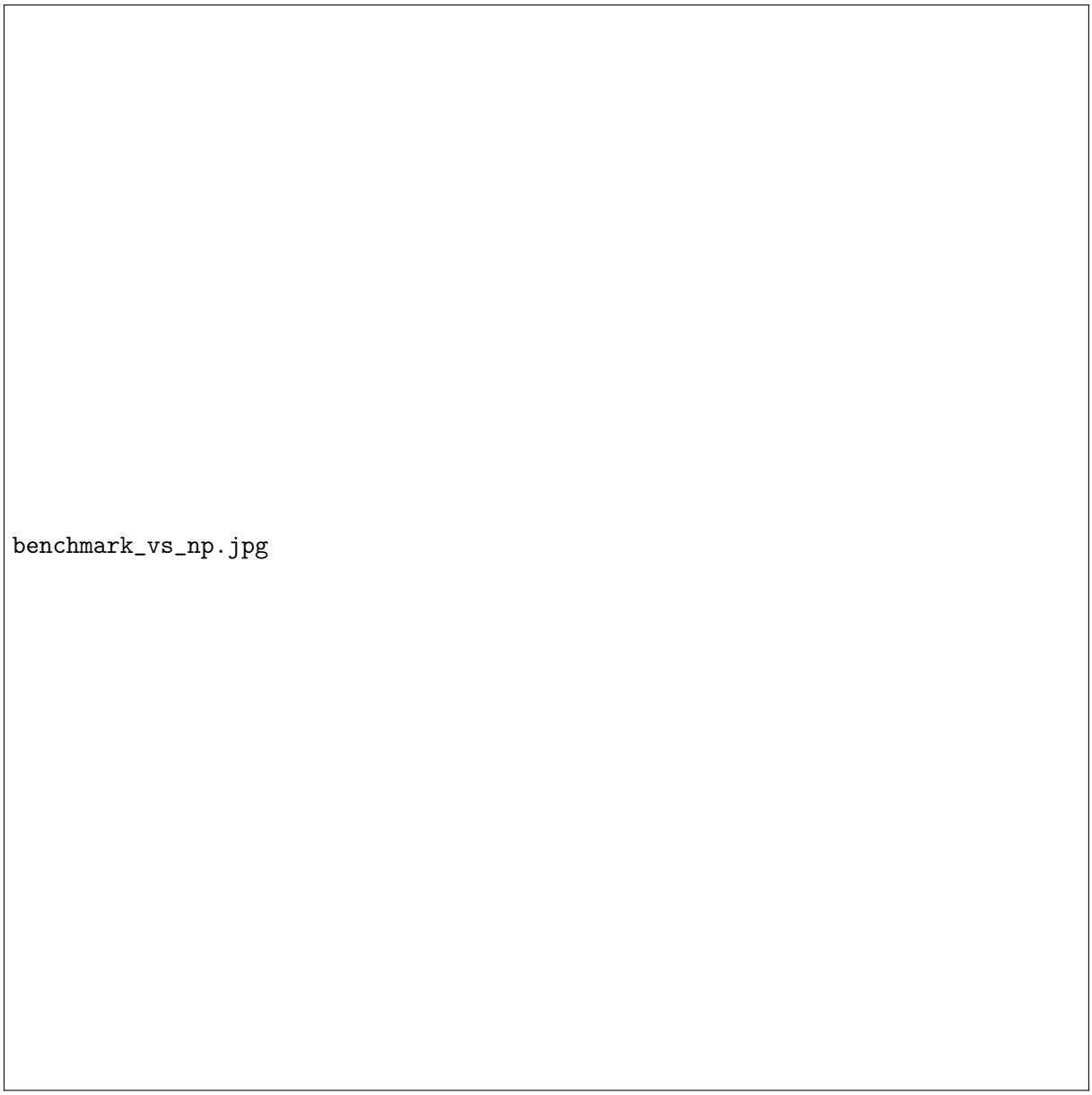

Figure 8: Strong scaling of the observable post-processing kernel.

from workers. For the observables workflow, the amount of useful work per process eventually becomes too small compared to the fixed costs of launching processes and reducing the final results.

Resource utilization is therefore just as important as raw parallelism. The orchestrator-worker design helps maintain high CPU occupancy by ensuring that free workers are immediately assigned new tasks from the queue. At the same time, the staged execution model avoids wasting resources on production jobs before equilibration has completed.

Combining MPI and OpenMP also requires careful control to avoid oversubscription. If too many MPI ranks each spawn too many OpenMP threads, the runtime can degrade due to thread collisions and operating system scheduling overhead. This is why the Makefile parameters `NP` and `OMPTHREADS` are treated as a coordinated resource-management mechanism rather than as independent tuning knobs.

observables_vs_np.jpg

Figure 9: Mean radius of gyration and end-to-end distance versus MPI process count.

Finally, the scaling trends also reflect the nonlinearity of Monte Carlo workloads themselves. Some replicas converge faster than others, some trajectories generate more favourable moved/-fixed partitions inside `delta_energy`, and some process counts match the available job granularity better than others. For this reason, the best-performing configuration is not always the one with the largest number of cores, but the one that best matches the size and structure of the workload.

10.4 Combined Parallel Workflow

The final program combines several levels of parallelism into a single coordinated workflow. At the top level, MPI is used to distribute work across processes in an orchestrator-worker star topology. Depending on the phase of execution, workers either participate in collaborative equilibration or execute independent production replicas drawn from a global queue.

Within each worker, OpenMP is used to accelerate the most expensive energy kernels, especially

the nested Lennard-Jones loops in `compute_total_energy` and `delta_energy`. This creates a hybrid structure in which MPI handles macro-level task distribution while OpenMP performs micro-level acceleration inside each simulation task.

A key design choice is that the simulation is separated into phases so that resources can be reused dynamically. If an equilibrated geometry already exists, the master can bypass the equilibration stage for that configuration and immediately unlock the corresponding production jobs. Meanwhile, workers that finish equilibration for other conformations are reassigned to the shared production queue, ensuring that CPU resources are not left idle.

This combined architecture does not simply attempt to maximize the number of active cores at all times. Instead, it tries to maximize the amount of useful work completed per unit time while respecting the constraints of communication overhead, thread contention, and workload granularity. In this sense, the program acts as a coordinated HPC ecosystem rather than as a single parallel loop.

References

- Chodera, J.D. (2016). A simple method for automated equilibration detection in molecular simulations. *J. Chem. Theory Comput.*, 12(4), 1799–1805. doi:10.1021/acs.jctc.5b00784
- Roy, V. (2020). Convergence Diagnostics for Markov Chain Monte Carlo. *Annual Review of Statistics and Its Application*, 7, 387–412. doi:10.1146/annurev-statistics-031219-041300
- Martin, M.G.; Siepmann, J.I. TraPPE-UA reference.
- Sæther et al. OPLS-AA effective polyethylene reference.

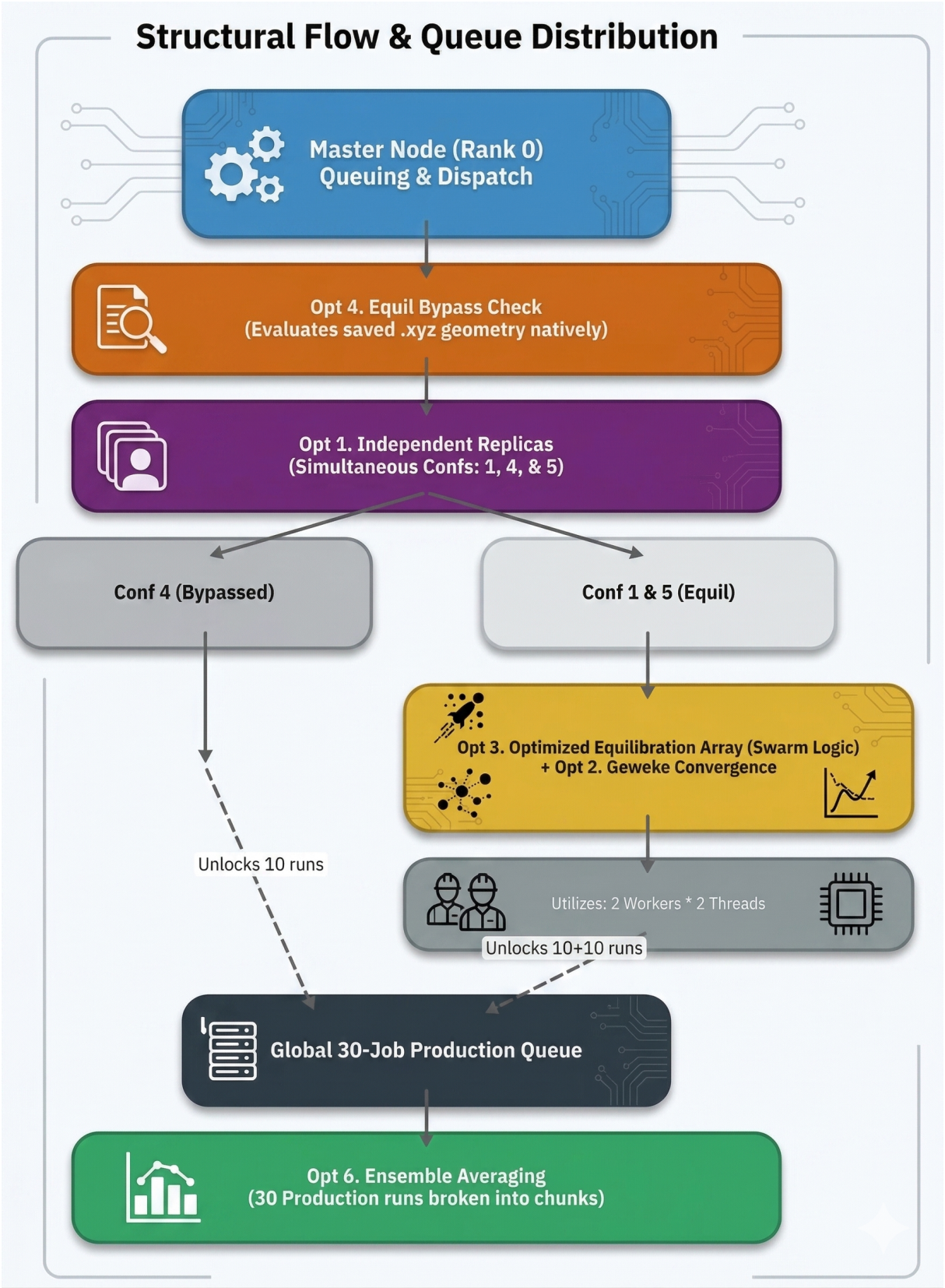

Figure 10: Schematic view of the combined MPI/OpenMP resource utilization strategy.